



SÃO
PAULO
TECH
SCHOOL

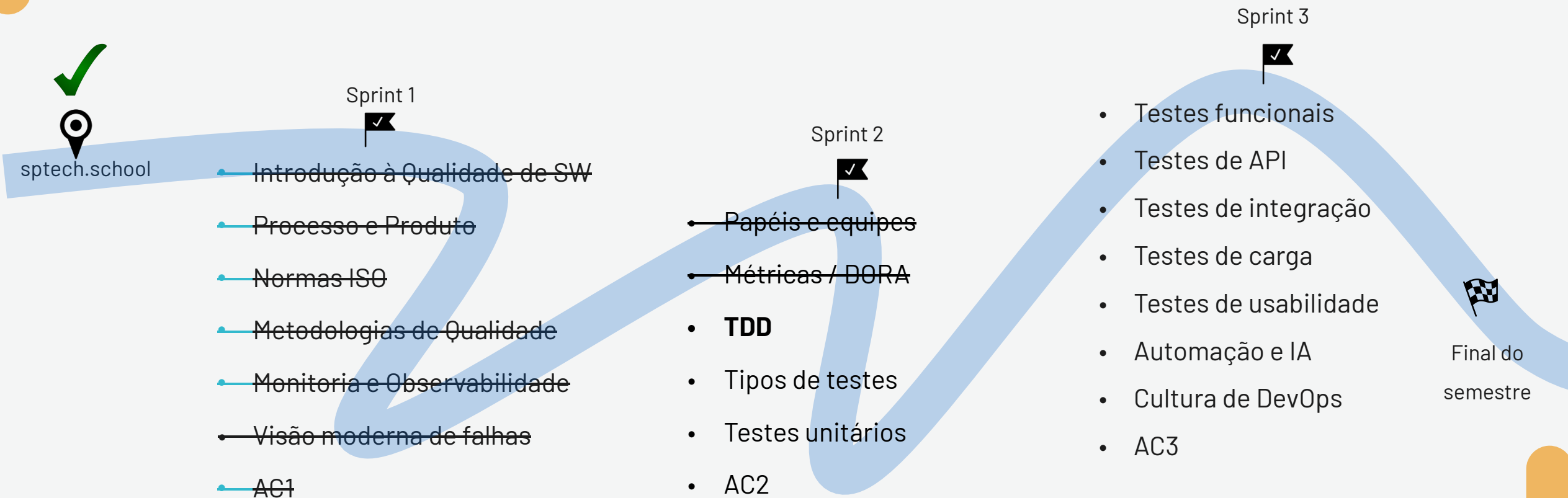
Qualidade de Software

TDD - Test Driven Development

24/mar

Prates

Nosso caminho - Ementa



Metodologias específicas para testes

TDD
BDD



Metodologias específicas para testes

Motivação e histórico

- Aumento da complexidade de sistemas
- Alternativas para reduzir a quantidade de problemas
- TDD se inicia em 2002, com Kent Beck
- Abordagem complementar a XP (Xtreme Programming)
- Foco em design orientado a testes e mudanças contínuas



Metodologias específicas para testes

Motivação e histórico

- BDD se inicia em 2003, com Dan North
- Evolução natural do TDD, focando nos usuários de negócios e não técnicos
- Enfatiza comportamento do sistema, não apenas funcionalidades técnicas

TDD - Test Driven Development

The background of the slide is a dark, almost black, field filled with intricate, flowing white patterns. These patterns consist of numerous thin, curved lines that sweep across the frame, creating a sense of dynamic movement. Interspersed among these lines are many small, bright white dots and star-like sparkles, some of which appear to be part of the flowing lines, while others are scattered independently. The overall effect is reminiscent of a cosmic nebula or a complex, abstract data visualization.



TDD - Test Driven Development

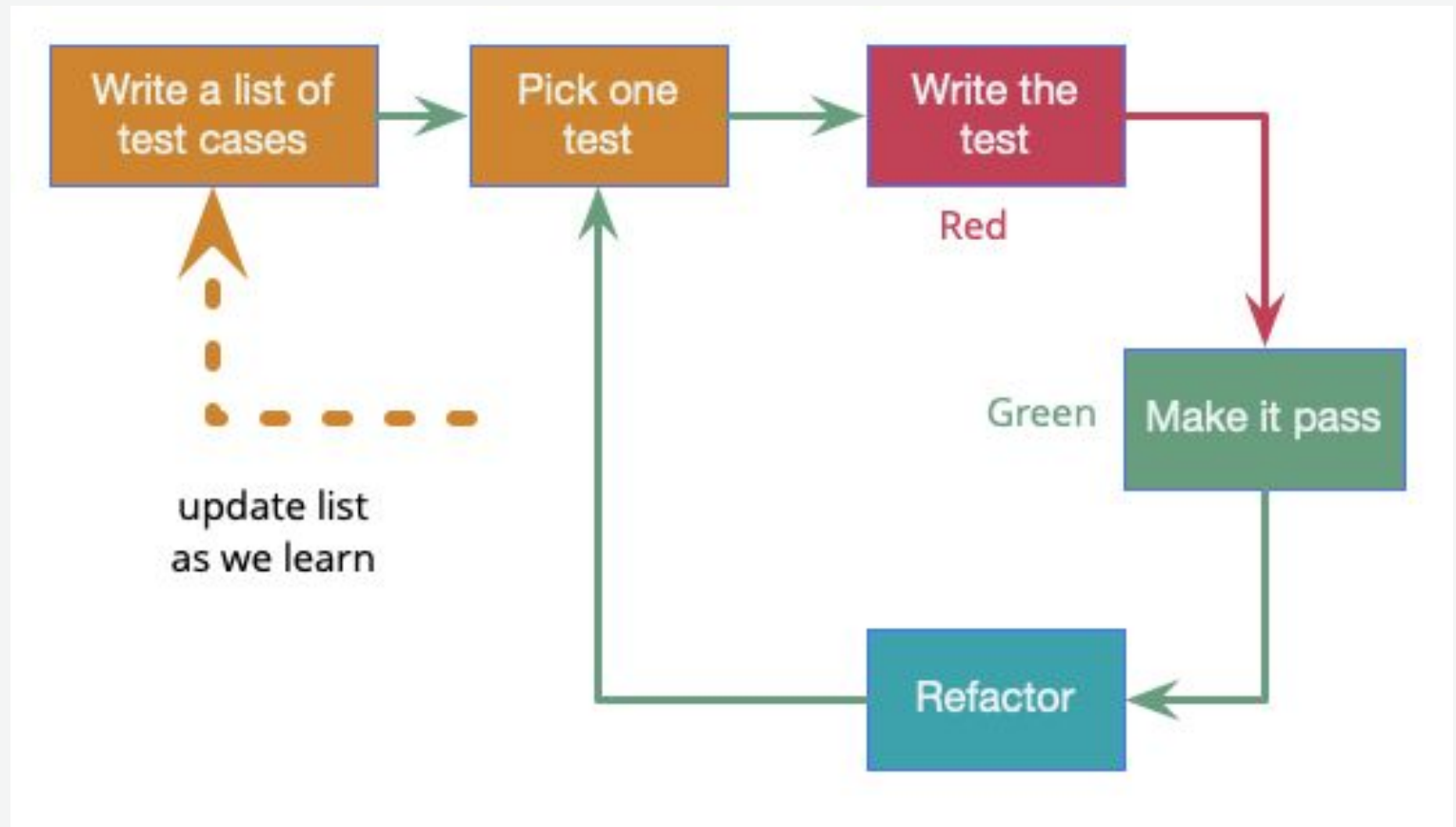
Conceito

- Escrever os testes **antes** do código de produção
- Processo cíclico: Red → Green → Refactor

Ciclo

- Escreva um teste que falha (Red)
- Escreva o código mínimo para passar o teste (Green)
- Refatore o código mantendo os testes verdes (Refactor)

TDD - Test Driven Development



TDD - Test Driven Development

Exemplo

```
def test_soma():  
    assert soma(2, 3) == 5  
  
test_soma()
```

```
rodrigoprates@MacBook-Pro-de-Rodrigo dev % python tdd1.py  
Traceback (most recent call last):  
  File "tdd1.py", line 4, in <module>  
    test_soma()  
  File "tdd1.py", line 2, in test_soma  
    assert soma(2, 3) == 5  
NameError: global name 'soma' is not defined
```

TDD - Test Driven Development

Exemplo

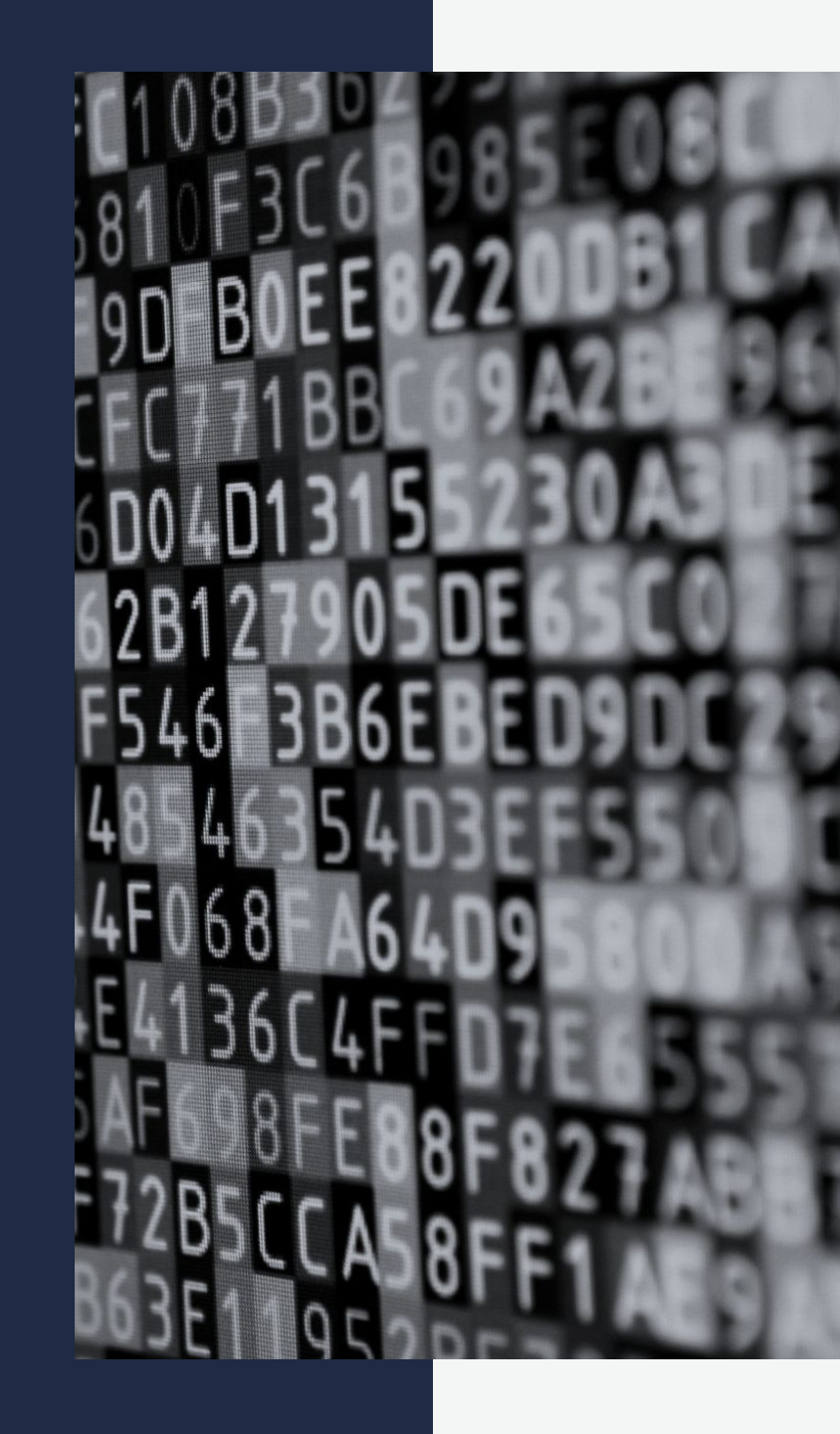
```
def test_soma():  
    assert soma(2, 3) == 5  
  
def soma(a, b):  
    return a + b  
  
test_soma()
```

TDD - Test Driven Development

Exemplo

```
def test_soma():  
    assert soma(2, 3) == 5  
  
def soma(a, b):  
    return a + b + 1  
  
test_soma()
```

```
[rodrigoprates@MacBook-Pro-de-Rodrigo dev % python tdd1.py  
Traceback (most recent call last):  
  File "tdd1.py", line 7, in <module>  
    test_soma()  
  File "tdd1.py", line 2, in test_soma  
    assert soma(2, 3) == 5  
AssertionError
```



TDD - Test Driven Development

Vantagens do TDD

- Design mais limpo e coeso
- Documentação viva do comportamento esperado
- Menor número de bugs em produção



TDD - Test Driven Development

Boas práticas para implementar

- Ferramentas: JUnit, pytest, Jest, etc.
- Cultura de testes como parte do fluxo de trabalho
- Feedback rápido via pipelines integrados
- Pair programming e code reviews com foco em testes

Exemplo com Pytest

```
rodrigoprates@MacBook-Pro-de-Rodrigo dev % pytest tdd1.py
===== test session starts =====
platform darwin -- Python 3.9.5, pytest-8.3.5, pluggy-1.5.0
rootdir: /Users/rodrigoprates/Desktop/SPTECH/dev
collected 0 items / 1 error

===== ERRORS =====
ERROR collecting tdd1.py
tdd1.py:7: in <module>
    test_soma()
tdd1.py:2: in test_soma
    assert soma(2, 3) == 5
E   assert 6 == 5
E   + where 6 = soma(2, 3)

===== short test summary info =====
ERROR tdd1.py - assert 6 == 5
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.20s =====
```

TDD - Test Driven Development

Exemplo com Pytest

```
[rodrigoprates@MacBook-Pro-de-Rodrigo dev % pytest tdd1.py]
===== test session starts =====
platform darwin -- Python 3.9.5, pytest-8.3.5, pluggy-1.5.0
rootdir: /Users/rodrigoprates/Desktop/SPTECH/dev
collected 1 item

tdd1.py . [100%]

===== 1 passed in 0.01s =====
```



TDD - Test Driven Development

Refatorar

```
from calc import soma

def test_soma_positivos():
    assert soma(10, 5) == 15

def test_soma_negativos():
    assert soma(-2, -3) == -5

def test_soma_misto():
    assert soma(-1, 4) == 3

def test_soma_zero():
    assert soma(0, 0) == 0
```



Atividade

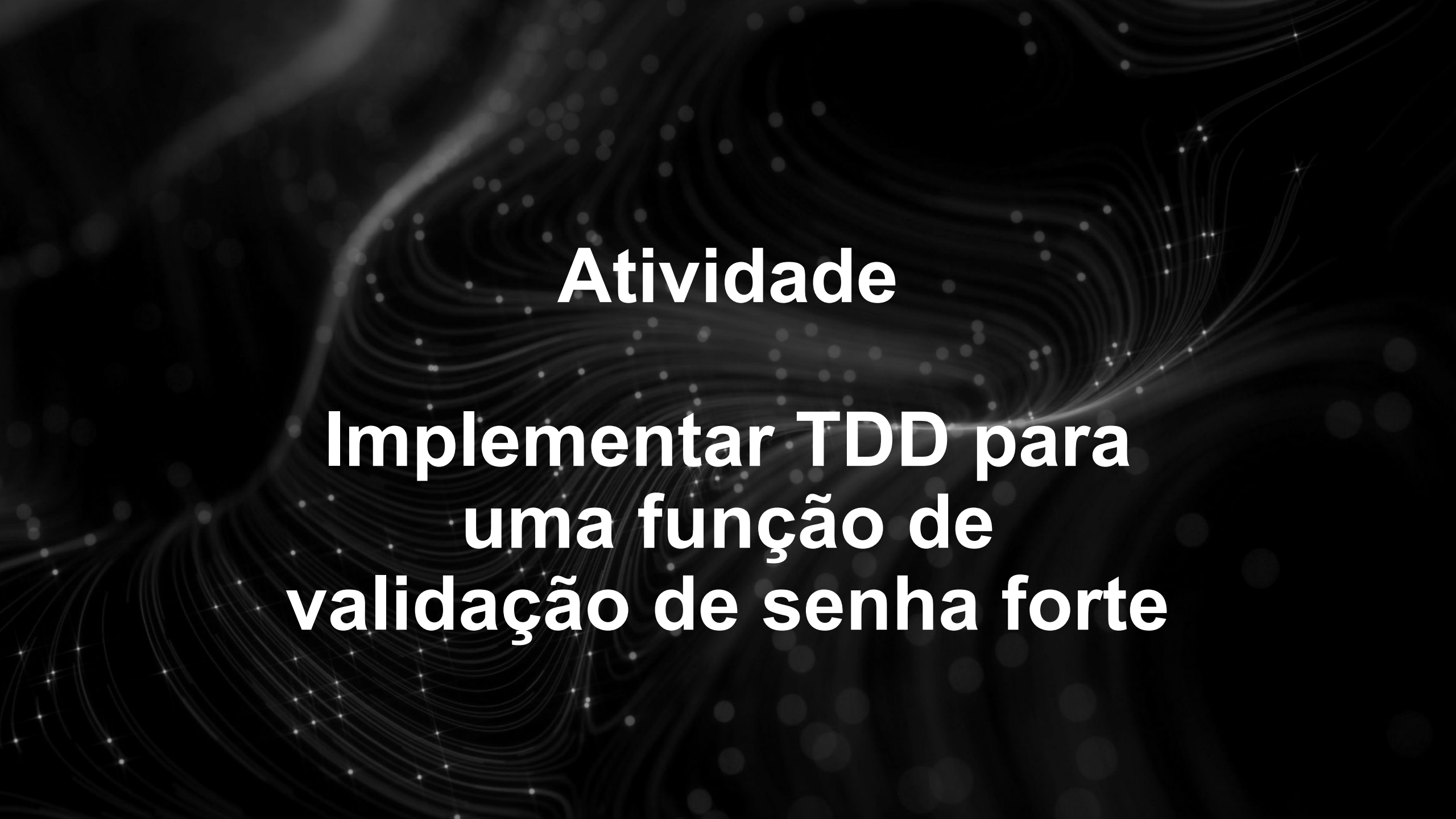
**Implementar TDD para
uma função de
validação de CPF**



Atividade - validação de CPF

Instruções

- Implemente uma função `valida_cpf(cpf: str)` que retorne `True` se o CPF for válido e `False` caso contrário
- Use o ciclo de TDD para isso:
 - Escreva um teste que falha
 - Implemente o código mínimo para passar o teste
 - Refatore se necessário
- Adicione mais testes para cobrir cenários relevantes



Atividade

**Implementar TDD para
uma função de
validação de senha forte**



Atividade - validação de senha forte

Instruções

- Implemente uma função `valida_senha(cpf: str)` que retorne `True` se a senha for válida ou `False` se não
- Use o ciclo de TDD para isso:
 - Escreva um teste que falha
 - Implemente o código mínimo para passar o teste
 - Refatore se necessário
- Adicione mais testes para cobrir cenários relevantes



Atividade - validação de senha forte

Regras para validação da senha

- A senha será considerada válida somente se:
 - tiver pelo menos 8 caracteres
 - possuir ao menos 1 letra maiúscula
 - possuir ao menos 1 letra minúscula
 - possuir ao menos 1 número
 - possuir ao menos 1 caractere especial
 - não contiver espaços em branco



Atividade - validação de senha forte

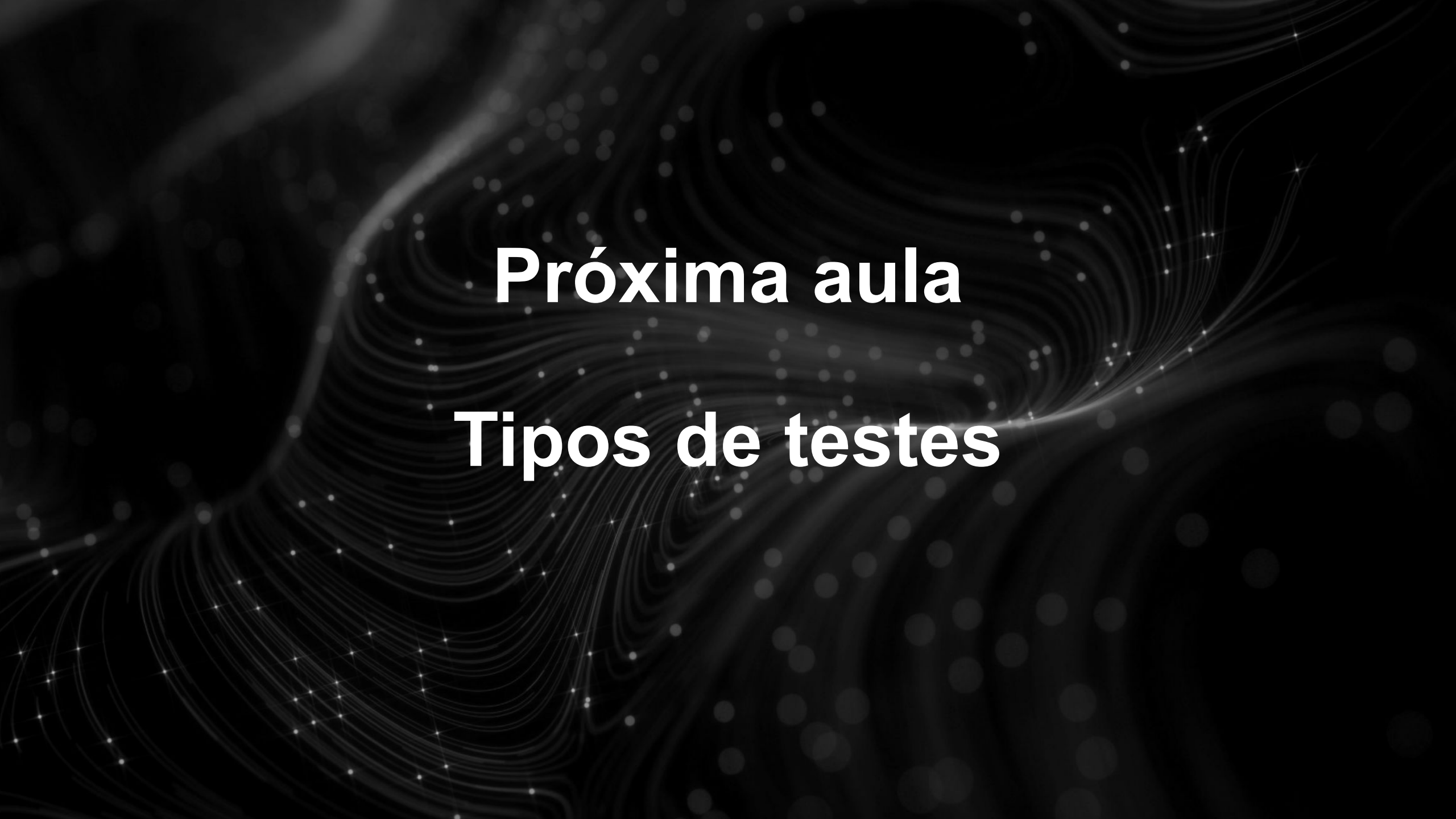
Casos de teste sugeridos

- Senha válida completa
- Senha com menos de 8 caracteres
- Senha sem letra maiúscula
- Senha sem letra minúscula
- Senha sem número
- Senha sem caractere especial
- Senha com espaço em branco
- Senha vazia

Referências

- TDD - <https://martinfowler.com/bliki/TestDrivenDevelopment.html>
- Kent Beck – Test-Driven Development: By Example
- BDD – Discovery: Explore behaviour using Examples
- BDD - <https://www.manning.com/books/bdd-in-action>

- Cucumber - <https://cucumber.io/docs/gherkin/reference>
- MochaJS - <https://mochajs.org/#run-cycle-overview>
- Behave - <https://behave.readthedocs.io/en/latest/>
- PyTest - <https://docs.pytest.org>

The background is a dark, almost black, space filled with intricate, glowing patterns. These patterns consist of numerous thin, curved lines that flow and swirl across the frame, creating a sense of movement and depth. Interspersed among these lines are many small, bright white dots of varying sizes, some of which have a soft, out-of-focus glow, resembling distant stars or bokeh light effects. The overall composition is dynamic and visually rich, providing a sophisticated backdrop for the text.

Próxima aula

Tipos de testes



Agradeço a sua atenção!

Qualidade de Software

Professor Prates

Aula 6

SÃO
PAULO
TECH
SCHOOL